

Custom Systems Atlas

Everything in `0_Scripts` that you wrote — what each system does, what it talks to, and where it will bite. Third-party assets are excluded. Every excerpt is copied from the branch.

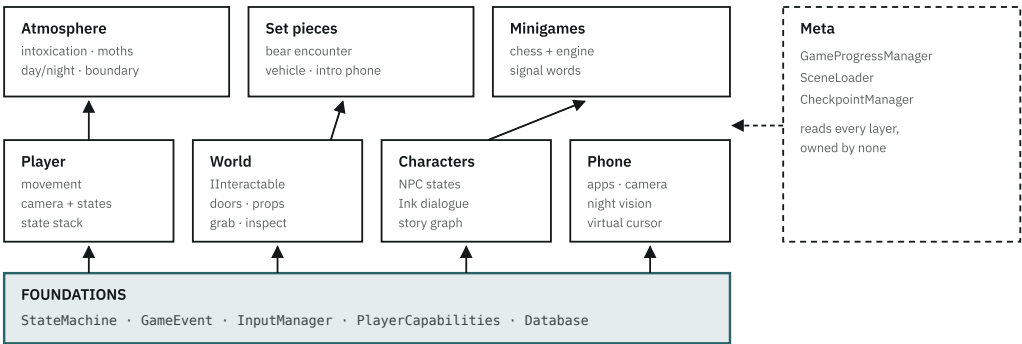
How to read this

Systems are grouped by the layer they sit in, largest first inside each layer.

Each entry gives you the same five things: what it does, where it lives, one excerpt worth seeing, what it connects to, and what to watch. The **Watch** line is only present where there is something real — absence of it means the system is in good shape.

LAYER	SYSTEMS	LINES
Foundations	StateMachine, GameEvent, InputManager, PlayerCapabilities, Database, ReferenceHub	~1,800
Player	Movement, CameraControllerNew, camera states, state stack, boundary, inventory, footsteps	5,012
World	Interaction, props, grabbing, inspection, plates, IK	4,650
Characters	NPC states, dialogue (Ink), story graph, QTE	5,085
The phone	Smartphone shell, apps, camera app, night vision, virtual cursor	5,195
Atmosphere	Intoxication, moth swarm, day/night, boundary distortion	3,592
Set pieces	Bear encounter, vehicle, intro/menu phone	4,440
Minigames	Chess + engine, Signal Words	8,923
Meta	Progression, scenes, checkpoints, audio	3,400

arrows point the way dependencies run — upward only, except Meta, which reaches across



The layering is real, not decorative: it matches the assembly definition graph. The one system that reaches across every layer is Meta, and that is exactly why progression and checkpoints were the fragile parts.

Foundations

Small, and everything else sits on them. A bug here is a bug everywhere.

StateMachine 160 lines · 6 files

0_Scripts/StateMachine/

A generic transition graph: states, predicates, and "any" transitions that can fire from wherever you are. The NPCs and the bear both run on it. Notably the *player* does not — the player uses a stack instead, because interruptions nest.

0_Scripts/StateMachine/StateMachine.cs

```
public void ChangeState(IState State)
{
    if (State == null) return;

    // This used to compare an IState against a StateNode. C# allows it, but it is
    // always false at runtime, so the guard never once fired and re-entering the
    // current state silently ran OnExit followed by OnEnter.
    if (CurrentState != null && ReferenceEquals(CurrentState.State, State)) return;

    var PrevState = CurrentState?.State;
    var NextNode = GetOrAddNode(State);

    PrevState?.OnExit();

    // Assigned BEFORE OnEnter, so a re-entrant ChangeState from inside OnEnter is
    // not overwritten by this assignment afterwards.
    CurrentState = NextNode;

    NextNode.State?.OnEnter();
}
```

USED BY NPCController, BearStateMachine

NOTE Both comments describe bugs that were already fixed before this branch. The ordering rule — assign before `OnEnter` — is the one worth keeping in mind when writing new states.

InputManager 574 lines · 4 files

0_Scripts/PlayerInput/ · [DefaultExecutionOrder(-200)]

Wraps the Input System behind named properties, so no gameplay script ever touches an `InputAction`. Every button is looked up by path and degrades to a no-op rather than throwing when an action is missing — and it warns once, not every frame.

```
0_Scripts/PlayerInput/InputManager.cs
```

```
public Vector2 Move => moveAction != null ? moveAction.ReadValue<Vector2>() : Vector2.zero;
public Vector2 Look => lookAction != null ? lookAction.ReadValue<Vector2>() : Vector2.zero;
```

```
public InputButton Run      => Button(PlayerMapName + "/Run");
public InputButton Interact => Button(PlayerMapName + "/Interact");
public InputButton Smartphone => Button(PlayerMapName + "/Smartphone");
```

CONNECTS Player movement and camera, the phone, the virtual cursor, dialogue advance, every minigame

NOTE One of the better-built systems. The duplicate-destroy race is handled correctly, and it is the model the other managers should follow.

GameEvent ~140 lines

0_Scripts/GameState/GameEvent.cs · GameEventListener.cs · GameEventTrigger.cs

A broadcast channel as a ScriptableObject asset. Listeners register while enabled; anything holding the asset can raise it without knowing who listens. Now walks a snapshot, because a listener disabling a GameObject used to throw mid-walk and skip everyone after it.

CONNECTS GameManager, trigger volumes, UI, the moth swarm's suppression hooks

WATCH **Only one channel exists so far.** The pattern scales well; the codebase mostly reaches through singletons instead, which is why there are still 248 `.Instance.` dereferences.

PlayerCapabilities 293 lines · 2 files

0_Scripts/PlayerState/

The single authority on what the player may do, built on this branch. Two contributors — the running player state (by replacement) and reference-counted holds (by union). Covered in full in the Systems Handbook.

CONNECTS PlayerMovement, CameraControllerNew, PlayerUI, PlayerInventory, the phone, QTEs, the bear kill, the checkpoint gallery

Database · ReferenceHub · GameManager ~500 lines

0_Scripts/GameState/ · Database runs at [DefaultExecutionOrder(-1000)]

Scene-reference lookup tables. Database holds transforms and destinations the story refers to by name; ReferenceHub holds the player head and camera; GameManager holds per-state spawn positions.

WATCH

Three overlapping registries. All three answer "where is the player / a thing", and Camera.main is a fourth route to the camera. Consolidating on ReferenceHub would remove a whole class of "which one is stale" bugs.

The player

5,012 lines across 41 files. The most heavily refactored part of the project.

PlayerMovement ~490 lines

0_Scripts/Player/PlayerControls/PlayerMovement.cs

A `CharacterController` driven by smoothed 2D velocity, with deliberate ground stickiness. It exposes readouts — `CurrentSpeed`, `NormalizedSpeed`, `RunBlend`, the animator's footstep curve — that the camera's head bob and the footstep audio both ride, so those can never drift out of phase with the feet.

TICKED BY `PlayerStateController.Update()`, exactly once a frame, whatever the state
GATED BY `PlayerCapabilities.CanMove` — one read, where there used to be two competing gates

CameraControllerNew + camera states ~1,900 lines

0_Scripts/Player/PlayerControls/CameraControllerNew.cs · Player/StateMachineCamera/ ·
`[DefaultExecutionOrder(-50)]`

Eight camera modes — `FreeLook`, `Dialogue`, `Sitting`, `Locked`, `HeadFollow`, `BoardGameTopDown`, `PhoneLook`, `Inventory` — each a `BaseCameraState`. The controller owns yaw, pitch, FOV and head bob; the states decide how the transform is driven.

Two mechanisms here are worth copying elsewhere, both keyed rather than boolean, because suppression nests:

```
0_Scripts/Player/PlayerControls/CameraControllerNew.cs

public void SuppressLook(object key)           // keyed: a jump and a dialogue can both hold
public void SetSensitivityMultiplier(object key, float multiplier)
```

CONNECTS Driven by `PlayerStateController` only; never switch it directly or the next profile undoes you

NOTE The legacy `CameraController` (654 lines, 0 references) was deleted on this branch. The bear's kill-cam was the last thing still pointing at it.

Player state stack ~600 lines

[0_Scripts/Player/StateMachinePlayer/](#)

Deliberately not a state machine. The player has no transitions — every change is an explicit call from a chair, a story action, a minigame — but interruptions *nest*: talking to someone while seated has to put you back in the chair. So: a stack, where frame 0 is the base and is never popped.

Suspended frames are re-validated before being restored, because the world moves on while they sit there:

[0_Scripts/Player/StateMachinePlayer/PlayerStateStack.cs](#)

```
private void DiscardStaleFrames()
{
    while (frames.Count > 1)
    {
        Frame frame = frames[frames.Count - 1];

        if (frame.State.Validate(frame.Payload, out string reason))
            return;

        Debug.LogWarning($"[PlayerStates] Dropping stale {frame.State.Name} frame while resuming: {reason}.");

        frames.RemoveAt(frames.Count - 1);
    }
}
```

NOTE Depth is capped at 8 with a warning, so something pushing without popping is noisy and bounded rather than growing forever.

PlayerInventory 399 lines

[0_Scripts/Player/PlayerInventory.cs](#) · [InventoryItem.cs](#)

Fixed slots holding `InventoryItem` components, each pointing at a live scene object and an overlay UI index.

WATCH **Not saveable as written.** Items hold `Transform ExistingGameObject` and `SelectedUITransform` — live scene references that cannot be serialised. Inventory needs stable item ids and a data model before it can join the checkpoint system.

Player boundary ~430 lines

`0_Scripts/Player/PlayerBoundary/`

Invisible walls that push back rather than stop dead. A zone clamps position, but also exposes a 0–1 band progress that drives a screen distortion as the player presses outward — so the edge of the level is communicated rather than just enforced.

```
0_Scripts/Player/PlayerBoundary/PlayerBoundaryZone.cs
```

```
public Vector3 ClampPosition(Vector3 worldPosition)
public float   EvaluateBandProgress(Vector3 worldPosition)
public float   EvaluateEffectIntensity(Vector3 worldPosition)
public Vector3 ResistMovement(Vector3 worldPosition, Vector3 worldMove)
```

CONNECTS `BoundaryDistortionDriver` at `[DefaultExecutionOrder(120)]`, which fires threshold events as the player pushes further

NOTE A genuinely good horror idea, cleanly built. Story actions set and clear boundaries per scene.

LAYER 3

The world

Interaction, props, and the procedural animation rig.

Interaction 3,515 lines · 28 files

0_Scripts/Interaction/ · detection lives in `PlayerUI`

One interface, implemented by doors, curtains, the fridge, the stove, windows, chairs, plates, food, grabbable items and NPCs:

0_Scripts/Interaction/IInteractable/IInteractable.cs

```
public interface IInteractable
{
    public float MinInteractionDistance { get; set; }
    bool Interact();

    void ToggleVisualOutline(bool Enable);

    public bool CanInteract { get; set; }
    public string InteractText { get; set; }
}
```

Detection is a `SphereCast` from the camera, run every frame by `PlayerUI` — which is also the HUD:

0_Scripts/PlayerUI/PlayerUI.cs

```
if (PlayerCapabilities.CanInteract)
{
    RaycastHit h;
    if (Physics.SphereCast(playerCamera.transform.position, 0.01f,
                          playerCamera.transform.forward, out h,
                          interactRange, interactableLayer, QueryTriggerInteraction.Ignore))
    {
        if (h.collider.TryGetComponent<IInteractable>(out IInteractable i))
        {
            if (i.CanInteract)
            {
                // ... distance check, then:
                CurrentInteractable = i;

                if (InteractBuffer.hasBuffer)
                {
                    InteractBuffer.Consume();
                    return i.Interact();
                }
            }
        }
    }
}
```

CONNECTS

Gated by `PlayerCapabilities.CanInteract` ; buffered through `TimedInputBuffer` so a press just before an object comes into range still lands

WATCH

Three things at once. The HUD, the interaction raycast and the prompt text all live in one 212-line class. Splitting the detection into an `InteractionProbe` would let the minigames and the phone reuse it. **Outlines are dead:** every `ToggleVisualOutline` call site in `PlayerUI` is commented out, so the interface member is implemented everywhere and invoked nowhere. **The radius is decorative** — `0.01f` makes this a ray with extra cost; raise it to `~0.15` to make small props easier to hit, or use `Raycast` .

Props — doors, curtains, fridge, stove, windows ~900 lines

[0_Scripts/Interaction/Door.cs](#) · [Curtains.cs](#) · [Fridge.cs](#) · [Stove/](#) · [SlidingWindow.cs](#)

All built the same way, which is what made them saveable with one component: a public bool for state, a DOTween animation on a transform, and an `Interact()` that toggles.

`0_Scripts/Interaction/Door.cs` — the shape every prop shares

```
public bool IsOpen = false;

moveTween = door.DOLocalRotate(new Vector3(0, RotationAmount, 0), Duration)
    .SetEase(EaseMode)
    .SetLink(door.gameObject);
```

CONNECTS `FlagSaveable` reads that bool by name and snapshots the pose, so none of these scripts needed editing to become saveable

NOTE All 78 tween statements across the project now carry `SetLink`, added on this branch.

Grabbing and inspection ~950 lines

[0_Scripts/Interaction/GrabbableItem.cs](#) (605) · [InspectableItem.cs](#) (341) · [InspectableItemManager.cs](#)

Held items follow a pickup point, accumulate release velocity for throwing, and can be restricted to placement-only by systems that own where things may be put down — `PlateStack` does this. Inspection zooms an item to the camera with reduced look sensitivity.

WATCH `Camera.main` every frame while an item is held, in `UpdateCurrentDropZone`. This is the third way the codebase finds the camera. Route it through `ReferenceHub`.

IK / procedural animation 1,135 lines · 10 files

[0_Scripts/IK/](#) — [Spider.cs](#) (375), [Player3D.cs](#), [SkinwalkerIK.cs](#), [ScanArms*.cs](#), [ShakeBody.cs](#)

Procedural limb placement: leg orbits that step when they drift too far from their target, a skinwalker pivot, arm scanning. Execution order is set explicitly across the group (`SkinWalkerPivot` at 1, `Spider` at 2) so the rig resolves in order.

WATCH **Ownership is unclear.** Ten MonoBehaviours with overlapping names — `IK`, `IK_Feet_Up`, `PositionFoot`, `ScanArms`, `ScanArmsAdvanced` — and no entry point saying which belong to a live creature. Before this becomes a second antagonist, check which are actually on a prefab and delete the rest.

Characters and story

5,085 lines. The narrative machinery, and the part most in need of a sequencing pass.

NPC 3,096 lines · 29 files

0_Scripts/NPC/

`Npc` is the façade — it implements `IInteractable` and holds the head transform, dialogue, path follower and controller. `NpcController` runs the shared `StateMachine` over states like Idle, Move, Talk, Sit, Stand, Interrupt, WaitForPlayer.

0_Scripts/NPC/NpcController.cs

```
public class NpcController : BaseNpcController
{
    public StateMachine stateMachine { get; private set; }

    private new void Update()
    {
        base.Update();
        stateMachine.Update();
    }

    void Any(IState to, IPredicate condition) => stateMachine.AddAnyTransition(to, condition);
```

CONNECTS `NpcDatabase` resolves NPCs by name for the dialogue camera and story actions;
`NpcHeadFocus` drives look-at rigs; `NpcPathFollower` handles navigation

WATCH `NpcStateController.cs` is 93% commented out and
`RefrigeratorInteractionState.cs` 89%. Both are abandoned earlier designs still sitting
in the folder, making the real controller harder to find. Git remembers them — delete.

Dialogue (Ink) 521 lines · 5 files

0_Scripts/Dialogue/

Wraps Ink. `DialogueManager` owns the story, types lines out character by character, and parses tags to drive the rest of the game — speaker, audio, emotion, NPC state change, destination. Those tags are broadcast as static events:

0_Scripts/NPC/NpcManager.cs — the consumer side

```
DialogueManager.OnAudioTagFound += DialogueManager_OnAudioTagFound;
DialogueManager.OnEmotionTagFound += DialogueManager_OnEmotionTagFound;
DialogueManager.OnNpcStateChangeFound += DialogueManager_OnNpcStateChangeFound;
```

CONNECTS `NpcManager` (tags → animation and audio), `PlayerStateController` (dialogue state),
`DialogueVariables` (Ink globals)

NOTE The singleton was fixed on this branch — it used to clear its own static from a public
`OnDisable`, so disabling the dialogue UI killed it permanently.

Story graph 1,989 lines · 35 files

0_Scripts/Story/ — 6 sub-assemblies

A node graph made of ScriptableObject. A `StoryNode` holds a list of `StoryAction` assets and the id of the next node; `StoryNodeRunner` plays them and chains onward. Actions cover NPC movement, dialogue, audio, rigs, look-at, player orientation, boundaries, phone messages, journal entries, QTEs, progress stage — and now checkpoints.

0_Scripts/Story/RootStoryAction/StoryNodeRunner.cs

```
public IEnumerator RunNodeParallel(StoryNode node)
{
    int completedActions = 0;
    int totalActions = 0;

    foreach (StoryAction action in node.actions)
    {
        if (action == null) continue;
        totalActions++;

        StartCoroutine(RunSingleAction(action, () =>
        {
            completedActions++;
            node.ActionsCompleted = completedActions; // writes to the ASSET
        }));
    }

    yield return new WaitUntil(() => completedActions ≥ totalActions);

    OnNodeCompleted?.Invoke(node);
}
```

WATCH

Parallel only. Every action in a node starts at once. An ordered beat — walk to the door, *then* speak, *then* the lights cut — has to be faked with per-action `Delay` values that need re-tuning whenever an animation length changes. For a horror game, where scares are timing, this is the authoring bottleneck. **And `ActionsCompleted` is runtime state on a shared asset** — it dirties the `.asset` file and lands in your commits.

The phone

5,195 lines. The player's main tool, and the most elaborate single object in the game.

Smartphone shell + BaseApp ~900 lines

`0_Scripts/Smartphone/Smartphone.cs` · `BaseApp.cs` · `AppPopup.cs` · `UINavigationController.cs`

The phone tweens between a stowed and a raised pose; apps derive from `BaseApp`, which handles the open/close animation, the back button and the top display. Apps are Chat, Notes, Gallery, Music, Camera and Light.

WATCH

`BaseApp` declares a private `Awake` that wires the open and close buttons. Unity invokes only the most-derived `Awake`, so any subclass adding one silently stops those buttons working. This nearly happened while fixing `LightApp` on this branch — the fix there captures its value lazily instead. Make `BaseApp.Awake` `protected virtual` so subclasses are forced to call `base`.

CameraApp + night vision 1,318 + 60 lines

`0_Scripts/Smartphone/CameraApp.cs` · `NightVisionLight.cs`

The single most sophisticated script in the project. A second camera renders the viewfinder to a `RenderTexture` at reduced quality, a shader bakes blur, night grade, grain and gamma into the shot on the GPU, and the readback is asynchronous where supported.

The night-vision idea is the best horror mechanic in the codebase: lights that *only exist inside the phone's camera*, so the player's own eyes never see them and nothing about their position is given away.

`0_Scripts/Smartphone/NightVisionLight.cs`

```
private void CaptureBaseState()
{
    if (captured) return;
    if (target == null) target = GetComponent<Light>();
    if (target == null) { /* warn */ return; }

    baseIntensity = target.intensity;
    target.enabled = false;    // invisible until a night-vision render asks for it
    captured = true;
}

private void OnEnable() { CaptureBaseState(); CameraApp.RegisterNightVisionLight(this); }
private void OnDisable() { CameraApp.UnregisterNightVisionLight(this); }
```

CONNECTS

Registers into `CameraApp`, which switches these on around the capture render and off again before the main camera draws

NOTE

It also drops LOD bias and disables shadows and the depth prepass for the capture — a second viewpoint has to be rasterised, but not at full quality. This is the pattern the checkpoint photographer follows.

Virtual cursor 489 + ~600 lines

[0_Scripts/VirtualCursor/](#) · [Smartphone/WorldCanvasVirtualCursor.cs](#) · [VirtualMouseFromMouse.cs](#)

A gamepad-drivable pointer for world-space canvases — the phone screen, the chess board, the word game. Raycasts UI itself rather than relying on the standard input module.

WATCH

Four cursor implementations. `VirtualCursor`, `VirtualCursorUIRaycaster`, `VirtualCursorWorldCanvasUIRaycaster`, `WorldCanvasVirtualCursor`, `VirtualMouseFromMouse`, plus a signal-words-specific input class. They overlap heavily. Pick one and delete the others before adding a third minigame.

Atmosphere

3,592 lines. The systems that do the actual frightening, and the best-written code in the project.

Intoxication 2,071 lines · 8 files

0_Scripts/Intoxication/ · [DefaultExecutionOrder(-100)]

The whole effect hangs off two floats. `Intoxication` is how drunk the player is; `Wrongness` blends a WARM preset toward a WRONG one. Nothing turns on or off between them, so `Wrongness` can be crept upward across an evening without the player catching a transition.

```
0_Scripts/Intoxication/IntoxicationController.cs
```

```
public float Intoxication ⇒ intoxication;
public float Wrongness    ⇒ wrongness;
public float Master        { get; private set; }
public DrunkPreset Current ⇒ blended;

public void SetIntoxication(float value);
public void SetWrongness(float value);
public Coroutine LerpIntoxication(float target, float duration);
public Coroutine LerpWrongness(float target, float duration);
```

Every oscillator integrates its own phase rather than sampling a global clock, so a period can change mid-scene without the waveform jumping. It drives double vision, barrel distortion, depth of field, chromatic aberration, lens breathing, vignette, grade and bloom — plus a "clarity flash" where everything snaps to zero for a moment, which in a permanently distorted frame is the most alarming thing available.

CONNECTS `IntoxicationCameraSway` at order 2000 (after everything else has moved the camera),
 `IntoxicationAudio`, `GhostLagEcho`

NOTE This is already a finished fear renderer that happens to represent alcohol. Feeding
 `Wrongness` from threat proximity and darkness instead is the single highest-value change
 available in this codebase.

Moth swarm 1,314 lines · 3 files

0_Scripts/Ambience/Moths/

Particle moths that orbit a light, scatter when disturbed, and stay off the camera lens. Runs in `LateUpdate` so the camera and the torch have finished moving before the swarm is asked where they are.

The suppression API is the pattern worth copying — counted, not boolean:

```
0_Scripts/Ambience/Moths/MothSwarmController.cs
```

```
/// Hold the swarm away for as long as `source` keeps asking. Counted, so several
/// overlapping zones can hold it at once and only the last one to release matters.
public void AddSuppressor(Object source)
public void RemoveSuppressor(Object source)

/// Parameterless so it can be wired straight into a UnityEvent.
public void Suppress() => latchedOff = true;
public void Allow()    => latchedOff = false;

public static void SuppressAll(); // static instance list, not a singleton
```

NOTE A static instance list rather than a singleton, so many swarms coexist and can be addressed as a group. This is the shape `PlayerCapabilities` holds were modelled on.

Day / night 207 lines · 4 files

0_Scripts/DayNight/

A `TimeService` advancing a `DateTime` at a multiplier, with `Observable<int>` minute and hour values firing change events. Drives sun and moon intensity and rotation.

CONNECTS The phone's clock reads `TimeManager.CurrentTime`; `TimeOfDaySaveable` captures and restores it

NOTE `SetTime` only ever moves forward. `RestoreTime` was added on this branch so loading an earlier checkpoint does not leave the sky where the player died.

Set pieces

Bear encounter 849 lines · 14 files

0_Scripts/BearEncounter/StateMachine/

Sleep → Wake → Stalk → fake charge or committed advance → Kill. The stalk state watches whether the player is looking at the bear and closes distance when they look away, which is the whole idea of the encounter.

0_Scripts/BearEncounter/StateMachine/BearWalkState.cs

```
void CheckIfPlayerIsLooking()
{
    if (BearAI.IsPlayerLookingAtBear())
        LookAwayTimer = 0f;
    else
        LookAwayTimer += Time.deltaTime;

    if (LookAwayTimer ≥ BearAI.BearStateData.MaxLookAwayTime)
    {
        BearAI.CloseDistanceState();
        LookAwayTimer = 0f;
    }
}
```

FIXED HERE

The stalk phase was being skipped entirely (`LookAwayTimer` seeded with `Time.time`); the kill-cam pointed at a deleted controller; perception measured the body's yaw instead of the camera; yaw-only turns went through quaternion component zeroing

WATCH

Four dead state files. `BearFakeChargeState` , `BearRoarState` , `BearRunState` , `BearDashState` are empty stubs, never instantiated. And two live class names read backwards relative to the enum — `BearAggressiveAdvanceState` is the fake charge, `BearChargeState` is the committed one. The mapping is correct; the names mislead.

Vehicle 1,972 lines · 6 files

0_Scripts/Vehicle/ · DriverSeatCamera at order 300

A raycast-wheel SUV with an actual gearbox — gear index, shift timer, RPM curve, engine audio driven by NormalizedEngineRpm . Input comes through an interface so a cutscene can drive it:

```
0_Scripts/Vehicle/ICarInput.cs
```

```
/// The five controls the SUV understands. Values are 0..1 so keyboard keys read as
/// on/off while gamepad triggers and sticks stay analogue.
/// Implement this to drive the car from a cutscene or a scripted intro sequence.
public interface ICarInput
{
    float Accelerate { get; }
    float SteerLeft  { get; }
    float SteerRight { get; }
    float Reverse    { get; }
    float Brake      { get; }
}
```

CONNECTS IntroCruiseInput at order -100 (reads fresh values before FixedUpdate),
TunnelLoopWarp at order 1000 (after the car has moved)

NOTE The execution-order comments here are exemplary — each one states what it must run before or after and why.

Intro / menu phone 1,619 lines · 8 files

0_Scripts/GameplaySpecific/

The main menu is a phone you record a clip on. MenuPhoneRecordSequence runs the ritual: press record, the HUD blinks, the feed freezes, a thumbnail flies into a Checkpoints icon, a toast confirms it saved.

CONNECTS onSavedToCheckpoints is the hook that starts the game — and now the natural place to open the real checkpoint gallery

NOTE Its own header used to say *"Nothing is written to disk. The clip is purely aesthetic."* The checkpoint system built on this branch is the real thing behind that fiction.

Minigames

8,923 lines — more than the player, the world and the phone combined would suggest.

Chess 7,764 lines · 34 files

0_Scripts/ChessMinigame/ · Engine/ (15 files, no UnityEngine references)

A real chess engine: bitboards, Zobrist hashing, a transposition table, iterative deepening, piece-square evaluation tables and an opening book. The engine layer has no Unity dependency at all, which makes it testable outside the editor — and a bridge translates:

0_Scripts/ChessMinigame/ChessEngineBridge.cs

```
/// Translates between the presentation types (Vector2Int squares, ChessPiece structs)
/// and the engine's compact representation. Everything Unity-facing goes through here,
/// which keeps the engine assembly free of UnityEngine references and therefore
/// testable outside the editor.
public static class ChessEngineBridge
{
    public static int ToSquare(Vector2Int coordinates)
    {
        if (!ChessUtility.IsInsideBoard(coordinates)) return -1;
        return coordinates.y * 8 + coordinates.x;
    }
}
```

The search runs off the main thread, with cancellation and a minimum think delay so the NPC does not answer instantly:

0_Scripts/ChessMinigame/NpcChessController.cs

```
private Task<ushort> RunSearchAsync(...)
{
    return Task.Run(() => ... );
}

private async Task WaitForMinimumDelay(float startTime, CancellationToken token)
```

Difficulty is a ScriptableObject with genuinely thoughtful knobs — think time, max depth, node cap, a centipawn margin for move choice, and a *slip chance* so the NPC occasionally plays a worse move rather than just searching shallower.

CONNECTS `SittingBoardGameState` + `BoardGameCameraState` for the camera; the virtual cursor for input; `PlayerGameState` for the cursor swap

WATCH **This is the largest system in your horror game, by a wide margin.** It is also the best-engineered. Nothing is wrong with it — but 7,764 lines is more than the player, the bear, the intoxication effect and the save system put together, and none of it makes the game frightening. Worth being deliberate about how much further it goes.

Signal Words 1,159 lines · 5 files

0_Scripts/Signal Words Minigame/

A Codenames-style team word game: a board of cards, a clue-giver that scores candidate clues against intended cards, and a blackout card that ends the round. Communicates entirely through typed UnityEvents, so the UI is fully decoupled:

```
0_Scripts/Signal Words Minigame/SignalWordTeamGameController.cs
```

```
public SignalWordTeamChangedEvent      OnTurnChanged;  
public SignalWordTeamClueChangedEvent  OnClueChanged;  
public SignalWordTeamCardRevealedEvent OnCardRevealed;  
public UnityEvent                      OnCorrectGuess;  
public UnityEvent                      OnWrongGuess;  
public UnityEvent                      OnBlackoutRevealed;  
public SignalWordTeamBoardCompletedEvent OnBoardCompleted;
```

NOTE The typed-UnityEvent approach here is cleaner than most of the codebase's singleton reach-through. Worth reusing.

Meta — progression, scenes, audio

Progression ~400 lines

0_Scripts/GameProgressManager.cs · GameProgressStage.cs · GameEventDefinition.cs · ProgressionEventManager.cs

A stage enum with gaps (`CabinArrival = 10` , `KitchenScene = 20` ...) so stages can be inserted later, plus per-event availability windows: an event is Locked before its `AvailableFrom` , Available inside the window, Consumed once used, Expired after.

FIXED HERE Every event asset was born `Expired` , because `ExpiresAt` defaults to `None` (0) and the test was a bare `>=` . `RestoreProgressStage` and event-state capture were added for checkpoints.

Scene management 234 lines · 3 files

0_Scripts/SceneManagement/

Scenes load in *groups* — a core scene plus lighting, UI, HUD and gameplay maps, each tagged by `SceneType` — with an async loading bar.

CONNECTS `CheckpointManager` stores the group index and reloads through this on load

WATCH Put `SceneLoader` in a bootstrap scene that is never itself reloaded. If it lives inside the group being reloaded it is destroyed mid-await and the load can hang.

Audio ~700 lines across 5 classes

GameAudioManager.cs · GameState/AudioManager.cs · DialogueAudioManager.cs · AudioZoneTrigger.cs · Player/IAudioManager.cs

`GameAudioManager` is the most complete — named `SoundID` fields for forest, cabin and cave ambience, dinner room tone, a tension drone, calm and tension music, QTE stings, doors, bathing — with zone and focus enums suggesting an intended crossfade model.

WATCH Four parallel paths, still roughly evenly used: `BroAudio` 22 call sites, `AudioManager.Instance` 22, raw `PlayOneShot` 18, `GameAudioManager.Instance` 7. There is no `AudioMixerGroup` routing anywhere, so there is no master volume, no ducking, no occlusion and no options menu. For a horror game this is the largest remaining architectural gap.

Making it error-free

Across the whole custom codebase, ordered by what costs you most. Save-system-specific items are in the Systems Handbook instead; these are the existing systems.

HIGHEST COST

One audio service on a mixer

Four paths, 69 call sites, no mixer routing. Until this is one service you cannot duck dialogue under ambience, drop the mix to near-silence before a scare, low-pass sound coming through a wall, or ship an options menu. Every one of those is something a horror game needs and none of them can be added per-call-site.

DO Pick BroAudio — you own it and it is the most capable. Route everything through mixer groups, then delete `AudioManager`, `GameAudioManager` and the unimplemented `IAudioManager` as you migrate.

HIGHEST COST

Sequential story actions

`RunNodeParallel` is the only mode. Ordered beats are faked with `Delay` floats that break whenever an animation length changes — and scares are timing, so this is the authoring bottleneck for every remaining scene.

DO Add a `SequenceAction` that runs a child list in order, and a `WaitForConditionAction` so a beat can wait on the player rather than a hardcoded delay.

WILL BITE

`StoryNode.ActionsCompleted` writes to the asset

A `ScriptableObject` field mutated at runtime. In the editor the value survives exiting play mode, dirties the `.asset`, and lands in your commits. If the same action asset ever appears in two nodes running at once they will also fight over it.

FIX Move the counter into `StoryNodeRunner`'s per-run state. Assets describe; runtime instances execute.

WILL BITE**BaseApp.Awake is private**

It wires the phone's open and close buttons. Unity calls only the most-derived `Awake`, so any app subclass that adds one silently breaks its own buttons — with no error, because nothing is wrong with either method.

FIX `protected virtual void Awake()`, and every subclass calls `base.Awake()`. One line now, versus a confusing bug later.

WILL BITE**QteStoryAction never completes**

The `QteManager.Instance.StartQTE` call is commented out, so `result` stays `None` and the `WaitUntil` waits forever — the story node hangs. The `try/finally` added on this branch stops it also stranding the player unable to move, but the node still never finishes.

FIX Finish `QteManager`, or give the wait a timeout the way `SaveCheckpointAction` does.

WILL BITE**Interaction outlines are implemented everywhere and called nowhere**

`IInteractable.ToggleVisualOutline` is on every interactable, and every call site in `PlayerUI.CheckInteract` is commented out. Players get a text prompt but no highlight on the object itself — which in a dark game is the feedback that matters most.

FIX Uncomment the calls and make sure the previous interactable is cleared when the raycast moves off it, which is what the commented block was attempting.

WILL BITE**Three registries and four routes to the camera**

`Database`, `ReferenceHub` and `GameManager` all answer "where is a thing", and the camera can be reached through `Camera.main`, `ReferenceHub.PlayerCamera`, `Player.Instance.cameraController` or a serialized field. `MenuPhoneCameraFeed` already carries a comment about `Camera.main` resolving to the wrong camera and breaking raycasts.

FIX Make `ReferenceHub` the one answer and route the rest through it. `GrabbableItem` calling `Camera.main` every frame while an item is held is the first one to change.

CLEANUP**Delete the dead files**

`NpcStateController.cs` (93% commented), `RefrigeratorInteractionState.cs` (89%), four empty bear state stubs, and roughly 4,600 commented-out lines project-wide. Git remembers all of it.

WHY Deleting them makes the remaining code searchable, which is worth more than it sounds when hunting a bug across 317 files.

CLEANUP**Consolidate the virtual cursors**

Five or six overlapping cursor classes across `VirtualCursor/`, `Smartphone/` and the word game. Each minigame added another.

FIX Settle on one before a third minigame needs one.

CLEANUP**Gate the logging**

369 `Debug.Log` calls with 5 `#if UNITY_EDITOR` guards — and that count went up on this branch, because the new systems log their failure paths. Error logs earn their place; the per-frame and per-event informational ones do not. String interpolation allocates even when the log is filtered.

FIX A static `GameLog` wrapper with `[Conditional("UNITY_EDITOR")]` on the informational methods strips them from builds at the call site, arguments included.

CLEANUP**Namespace the assemblies**

33 assembly definitions but only ~58 of 317 files declare a namespace, so almost everything shares the global namespace. `GameState` is both an enum and a folder; two camera controllers and four audio managers competed for the same mental space.

FIX Set `rootNamespace` on each `asmdef` and let the IDE move files into it. The assembly boundaries are already correct — this just makes them visible in code.

What is genuinely good and should not be touched: the assembly definition graph, `IntoxicationController`, `MothSwarmController`, `NightVisionLight`, `InputManager`, the player state stack, the chess engine's separation from Unity, and the execution-order comments in the vehicle code. Those are the standard the rest of the codebase should be brought up to.

317 custom scripts · 35,191 lines · 27 systems · branch `claude/horror-game-scripts-review-3eieth`